Kotlin One-Liners & Chain Functions

Master Functional Programming with Expressive Code

Discover the power of Kotlin's functional programming: chain multiple operations, transform data elegantly, and write expressive one-liners. This guide covers collections, strings, data transformations, functional patterns, conditional logic, advanced chains, practical use cases, and scope function mastery - all in concise, readable code.

COLLECTION OPERATIONS

```
listOf(1,2,3,4,5).map { it * 2 }.filter { it > 5 }.sum()
```

Transform, filter, and sum in one chain

```
// 18 (6+8+10)
```

```
listOf("apple","banana","cherry").filter { it.length > 5 }.map { it.uppercase() }.joinToString()
```

Filter by length, uppercase, join to string

```
// BANANA, CHERRY
```

```
listOf(1,2,3,4,5).takeWhile { it < 4 }.map { it * it }.reduce { acc, i -> acc + i }
```

Take while condition, square, then sum with reduce

```
// 14 (1+4+9)
```

```
(1..10).filter { it % 2 == 0 }.map { it * 3 }.take(3).toList()
```

Even numbers, triple them, take first 3

```
// [6, 12, 18]
```

```
listOf(1,2,3).flatMap { i -> listOf(i, i*10) }.sorted().reversed()
```

FlatMap to duplicate with multiplier, sort descending

```
// [30, 20, 10, 3, 2, 1]
```

```
listOf("a","bb","ccc").groupBy { it.length }.mapValues { it.value.size }
```

Group by string length, count each group

```
// {1=1, 2=1, 3=1}
```

```
listOf(1,2,3,4,5).windowed(3).map { it.sum() }.max()
```

Sliding window of 3, sum each, find max

```
// 12 (3+4+5)
```

```
listOf(1,2,2,3,3,3).groupingBy { it }.eachCount().maxByOrNull { it.value }?.key
```

Find most frequent element

```
// 3
```

```
(1..100).filter { it % 3 == 0 || it % 5 == 0 }.sum()
```

Sum of multiples of 3 or 5 up to 100

```
// 2418
```

```
listOf(1,2,3).associateWith { it * it }.filterValues { it > 4 }
```

Create map with squares, filter values > 4

```
// {3=9}
```

STRING MANIPULATION

```
"hello world".split(" ").joinToString("-") { it.uppercase() }
```

Split, uppercase each word, join with dash

```
// HELLO-WORLD
```

```
"The Quick Brown Fox".filter { it.isUpperCase() }.lowercase()
```

Extract uppercase letters, then lowercase them

```
// tqbf
```

```
"kotlin".mapIndexed { i, c -> if (i % 2 == 0) c.uppercase() else c }.joinToString("")
```

Uppercase every other character

```
// KoTlIn
```

```
"abcdef".chunked(2).joinToString(",") { it.reversed() }
```

Split into pairs, reverse each, join

```
// ba,dc,fe
```

```
"hello".toList().shuffled().joinToString("").also { println(it) }
```

Shuffle characters in string

```
// Random: olehl
```

```
"a1b2c3".partition { it.isDigit() }.let { "${it.first}-${it.second}" }
```

Separate digits and letters

```
// 123-abc
```

```
"Hello World".count { it.isUpperCase() }.let { "$it uppercase letters" }
```

Count uppercase letters with message

```
// 2 uppercase letters
```

```
"racecar".let { it == it.reversed() }
```

Check if palindrome in one line

```
// true
```

```
" trim me ".trim().replace(" ", "_").uppercase()
```
Chain trim, replace spaces, uppercase
```
// TRIM_ME
```

```
"abc123def456".filter { it.isDigit() }.map { it.digitToInt() }.sum()
```
Extract digits, convert to int, sum
```
// 21
```

DATA TRANSFORMATION

```
data class User(val name: String, val age: Int) listOf(User("Alice",25), User("Bob",30)).maxByOrNull { it.age }?.name
```
Find oldest user's name
```
// Bob
```

```
listOf(User("Alice",25), User("Bob",30)).partition { it.age >= 30 }.let { it.first.size to it.second.size }
```
Count adults and minors
```
// (1, 1)
```

```
listOf(User("Alice",25), User("Bob",30)).associateBy { it.name.lowercase() }.mapValues { it.value.age }
```
Create map of lowercase names to ages
```
// {alice=25, bob=30}
```

```
listOf(1 to "a", 2 to "b", 3 to "c").toMap().filterKeys { it > 1 }
```
Convert pairs to map, filter by keys
```
// {2=b, 3=c}
```

```
listOf("a" to 1, "b" to 2, "a" to 3).groupBy({ it.first }, { it.second }).mapValues { it.value.sum() }
```
Group by key and sum values
```
// {a=4, b=2}
```

```
listOf(1,2,3).zip(listOf("a","b","c")).map { "${it.first}${it.second}" }
```
Zip numbers with letters, concatenate
```
// [1a, 2b, 3c]
```

```
listOf(Triple(1,"a",true), Triple(2,"b",false)).filter { it.third }.map { it.second }
```
Filter triples by boolean, extract middle
```
// [a]
```

```
(1..5).map { it to it*it }.toMap().entries.sortedByDescending { it.value }.take(3)
```
Create number to square map, sort by value desc, take 3
```
// [5=25, 4=16, 3=9]
```

```
listOf("apple","banana","apricot").groupBy { it[0] }.mapKeys { it.key.uppercase() }
```
Group by first letter, uppercase the keys
```
// {A=[apple, apricot], B=[banana]}
```

```
listOf(1,2,3,4).fold("Result:") { acc, i -> "$acc $i" }.trim()
```
Fold list into string with accumulator
```
// Result: 1 2 3 4
```

FUNCTIONAL PROGRAMMING

```
listOf(1,2,3).map { it * 2 }.let { doubled -> doubled.sum() to doubled.average() }
```
Calculate sum and average of doubled values
```
// (12, 4.0)
```

```
5.let { it * it }.also { println("Square: $it") }.let { it + 10 }
```
Chain let and also for computation with logging
```
// 35 (also prints Square: 25)
```

```
"hello".run { uppercase() }.run { reversed() }.run { plus("!") }
```
Chain multiple run operations
```
// OLLEH!
```

```
listOf(1,2,3,4,5).takeIf { it.size > 3 }?.filter { it % 2 == 0 } ?: emptyList()
```
Conditional processing with takeIf and elvis
```
// [2, 4]
```

```
generateSequence(1) { it + 1 }.take(10).filter { it % 2 == 0 }.sum()
```
Infinite sequence, take 10, filter evens, sum
```
// 30
```

```
(1..100).asSequence().filter { it % 2 == 0 }.map { it * it }.take(5).toList()
```
Lazy sequence for performance
```
// [4, 16, 36, 64, 100]
```

```
listOf(1,2,3).fold(1) { acc, i -> acc * i }.let { "Factorial: $it" }
```

Calculate factorial using fold

```
// Factorial: 6
```

```
listOf("a","b","c").runningFold("") { acc, s -> acc + s }.drop(1)
```

Running fold to show intermediate results

```
// [a, ab, abc]
```

```
(1..10).partition { it % 2 == 0 }.let { (evens, odds) -> evens.sum() to odds.sum() }
```

Sum evens and odds separately

```
// (30, 25)
```

```
listOf(1,2,3).mapNotNull { if (it > 1) it * 2 else null }.sum()
```

Map with null filtering

```
// 10
```

CONDITIONAL OPERATIONS

```
listOf(1,2,3,4,5).firstOrNull { it > 3 }?.let { "Found: $it" } ?: "Not found"
```

Find first match with elvis operator fallback

```
// Found: 4
```

```
(1..10).count { it % 2 == 0 }.takeIf { it > 3 }?.let { "Many: $it" } ?: "Few"
```

Count with conditional message

```
// Many: 5
```

```
listOf(1,2,3,4,5).all { it > 0 } && listOf(2,4,6).all { it % 2 == 0 }
```

Combine multiple predicates

```
// true
```

```
listOf(1,2,3,4,5).any { it > 3 }.let { if (it) "Has large" else "All small" }
```

Any with conditional string

```
// Has large
```

```
(0..100).filter { it % 3 == 0 && it % 5 == 0 }.take(3)
```

FizzBuzz numbers (divisible by both)

```
// [0, 15, 30]
```

```
listOf(1,2,3,null,5).filterNotNull().takeWhile { it < 4 }.sum()
```

Remove nulls, take while condition, sum

```
// 6
```

```
"admin".let { it == "admin" || it == "root" }.let { if (it) "Access granted" else "Denied" }
```

Role check with boolean logic

```
// Access granted
```

```
listOf(10,20,30).maxOrNull()?.let { it > 25 }?.let { "Large max" } ?: "Small max"
```

Chain nullable operations

```
// Large max
```

```
(1..20).filter { it.toString().contains('1') }.count()
```

Count numbers containing digit 1

```
// 11
```

```
listOf("apple","banana").none { it.length > 10 }.let { if (it) "Valid" else "Invalid" }
```

Validate all items with none

```
// Valid
```

ADVANCED CHAINING

```
listOf(1,2,3).map { it to it*it }.toMap().entries.associate { it.key to it.value.toString() }
```

Create map, transform to string values

```
// {1=1, 2=4, 3=9}
```

```
(1..5).flatMap { i -> (1..i).map { j -> i to j } }.filter { it.first > it.second }
```

Generate pairs where first > second

```
// [(2,1), (3,1), (3,2), (4,1), ...]
```

```
listOf("a","bb","ccc").sortedBy { it.length }.reversed().withIndex().associate { it.value to it.index }
```

Sort, reverse, create index map

```
// {ccc=0, bb=1, a=2}
```

```
(1..10).chunked(3).map { it.average() }.map { it.toInt() }
```

Chunk into groups, average each, convert to int

```
// [2, 5, 8, 10]
```

```
listOf(1,2,3,4,5).scan(0) { acc, i -> acc + i }.zipWithNext().map { it.second - it.first }
```
Running sum, then differences between consecutive
```
// [1, 2, 3, 4, 5]
```

```
"abcdefgh".chunked(2).mapIndexed { i, s -> if (i % 2 == 0) s.uppercase() else s }.joinToString("")
```
Chunk pairs, alternate uppercase
```
// ABcdEFgh
```

```
listOf(1,2,3,4,5).windowed(2).map { it[0] * it[1] }.reduce { acc, i -> acc + i }
```
Multiply consecutive pairs, sum products
```
// 40 (2+6+12+20)
```

```
(1..100).filter { n -> (2 until n).none { n % it == 0 } }.take(10)
```
First 10 prime numbers
```
// [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

```
listOf("apple","banana","apricot").groupBy { it[0] }.mapValues { it.value.maxByOrNull { s -> s.length } }
```
Group by first letter, find longest in each
```
// {a=apricot, b=banana}
```

```
(1..5).map { i -> (1..i).toList() }.flatten().groupingBy { it }.eachCount()
```
Pyramid of numbers, count frequencies
```
// {1=5, 2=4, 3=3, 4=2, 5=1}
```

PRACTICAL ONE-LINERS

```
listOf("john.doe@email.com","jane@test.com").filter { it.matches(Regex(".*@.*\\..*")) }.map { it.substringBefore('@') }
```
Extract valid email usernames
```
// [john.doe, jane]
```

```
"192.168.1.1".split(".").map { it.toInt() }.all { it in 0..255 }
```
Validate IP address format
```
// true
```

```
listOf(10.5, 20.3, 30.1).map { "$%.2f".format(it) }.joinToString(", ")
```
Format currency with 2 decimals
```
// $10.50, $20.30, $30.10
```

```
"hello-world-kotlin".split("-").joinToString("") { it.replaceFirstChar { c -> c.uppercase() } }
```
Convert kebab-case to PascalCase
```
// HelloWorldKotlin
```

```
listOf("apple","APPLE","Apple").distinctBy { it.lowercase() }
```
Case-insensitive distinct
```
// [apple]
```

```
(1..12).groupBy { if (it <= 3) "Q1" else if (it <= 6) "Q2" else if (it <= 9) "Q3" else "Q4" }
```
Group months into quarters
```
// {Q1=[1,2,3], Q2=[4,5,6], Q3=[7,8,9], Q4=[10,11,12]}
```

```
"The quick brown fox".split(" ").map { it.length }.let { "Min: ${it.minOrNull()}, Max: ${it.maxOrNull()}" }
```
Find min and max word lengths
```
// Min: 3, Max: 5
```

```
listOf("user1","admin","user2","superadmin").partition { it.contains("admin") }.first
```
Extract all admin users
```
// [admin, superadmin]
```

```
System.currentTimeMillis().let { (it / 1000).toInt() }.toString()
```
Get Unix timestamp in seconds
```
// Current timestamp string
```

```
listOf(1,2,3,4,5).joinToString(separator = ",", prefix = "[", postfix = "]", limit = 3, truncated = "...") { "num$it" }
```
Advanced joinToString with all options
```
// [num1,num2,num3,...]
```

SCOPE FUNCTION MASTERY

```
"test".let { it.uppercase() }.also { println(it) }.run { length }.toString()
```
Chain let, also, run for transformation pipeline
```
// 4 (also prints TEST)
```

```
mutableListOf(1,2,3).apply { add(4); add(5) }.also { it.shuffle() }.sum()
```
Apply to modify, also to shuffle, then sum
```
// 15
```

```
StringBuilder().apply { append("Hello"); append(" "); append("World") }.toString().let { it.split(" ") }
```
Build string with apply, split with let
```
// [Hello, World]
```

```
listOf(1,2,3).takeIf { it.size > 2 }?.run { map { it * 2 } } ?: emptyList()
```
Conditional processing with takeIf and run
```
// [2, 4, 6]
```

```
"kotlin".run { this.uppercase() }.run { reversed() }.also { println("Result: $it") }
```
Multiple run chains with also for logging
```
// NILTOK (also prints)
```

```
5.let { x -> 10.let { y -> x * y } }.toString()
```
Nested let for scoped calculations
```
// 50
```

```
mutableMapOf<String, Int>().apply { put("a", 1); put("b", 2) }.entries.joinToString { "${it.key}=${it.value}" }
```
Apply to build map, then format entries
```
// a=1, b=2
```

```
listOf(1,2,3,4,5).run { filter { it % 2 == 0 } }.run { map { it * 3 } }.sum()
```
Multiple run scopes for transformation
```
// 18
```

```
"test".takeUnless { it.isEmpty() }?.uppercase() ?: "EMPTY"
```
TakeUnless for inverse condition check
```
// TEST
```

```
listOf(1,2,3).let { list -> list.sum() to list.average() }.let { "Sum: ${it.first}, Avg: ${it.second}" }
```
Multiple let scopes with pair
```
// Sum: 6, Avg: 2.0
```